

Cadence

Diseño e Implementación de un Lenguaje

1. Equipo

Nombre	Apellido	Legajo	E-mail
Felipe	Villanueva	65250	<i>fvillanueva@itba.edu.ar</i>
Tadeo	Gorganchian	65507	<i>tgorganchian@itba.edu.ar</i>
Eduardo	Tormakh	65155	<i>etormakh@itba.edu.ar</i>

2. Repositorio

La solución y su documentación serán versionadas en: <https://github.com/fvillanueva1/tpe-tla>

3. Dominio

3.1. Descripción

La música occidental tonal se apoya en un cuerpo de reglas conocido como armonía: un sistema que determina qué notas suenan bien simultáneamente, cómo se agrupan en acordes, cómo esos acordes se encadenan para producir tensión y resolución, y cómo se distribuyen entre las voces que los ejecutan.

Un músico formado no piensa en notas sueltas. No dice "*do, mi, sol*", dice "*la tónica*". No dice "*do mayor, sol mayor, la menor, fa mayor*", dice "*uno, cinco, seis, cuatro*" — una progresión que sostiene cientos de canciones populares. Esa capa de abstracción no es una comodidad de notación: es el vocabulario con el que los músicos efectivamente se comunican, y es enteramente mecánica. Dada una tonalidad y un grado, las notas del acorde quedan unívocamente determinadas.

Las herramientas de software para escribir música, sin embargo, operan casi siempre en el nivel más bajo posible: se ubican notas individuales sobre una grilla o un pentagrama. La abstracción con la que el músico piensa se pierde en la traducción, y toda operación de alto nivel — modular a otra tonalidad, sustituir un acorde por su relativo menor, redistribuir un acorde entre cuatro voces sin violar las reglas del contrapunto — se vuelve trabajo manual, repetitivo y propenso a errores.

Cadence es un lenguaje de dominio específico que invierte esa relación. El programador declara tonalidades, construye acordes por su grado dentro de ellas, encadena progresiones, las modula y las

distribuye entre voces; el compilador expande esas construcciones a las notas concretas, verifica que el resultado sea armónicamente correcto, y emite un archivo MIDI reproducible o una partitura.

El dominio es particularmente apto para la construcción de un compilador porque la teoría armónica es formal, mecánica y verificable. Existen notas que no existen, intervalos que son imposibles por definición, grados que se salen de una escala, acordes ajenos a la tonalidad en que fueron declarados, y — de manera más interesante — movimientos entre voces que la práctica común prohíbe explícitamente, como las quintas y octavas paralelas. Todas esas condiciones son decidibles estáticamente, lo que permite que la fase de análisis semántico rechace programas *musicalmente* incorrectos, y no meramente mal tipados.

Toda la ejecución ocurre dentro de la memoria del programa. No hay acceso a hardware de audio ni a sintetizadores externos: los artefactos generados son archivos que luego pueden reproducirse o imprimirse con herramientas de terceros.

3.2. Diferenciación respecto de trabajos previos

El dominio musical ha sido abordado en varias oportunidades. Los trabajos consultados construyen la pieza de abajo hacia arriba: nota → melodía o patrón → pista → composición. La nota es el tipo primitivo y todo lo demás es concatenación.

Cadence construye de arriba hacia abajo: tonalidad → grado → acorde → progresión → voces. La nota no es lo que el programador escribe, sino lo que el compilador deduce. De esa inversión se desprenden cuatro capacidades que no están presentes en los trabajos previos:

1. **El acorde se construye por grado, no por enumeración.** Trabajos anteriores permiten notas relativas a una escala (`degree(1, 3)`) o acordes con calidad explícita (`Cmaj7`), pero en el primer caso la construcción es monofónica y en el segundo el acorde es un literal absoluto, sin tonalidad de referencia ni función armónica asociada.
2. **La progresión es un tipo de dato de primera clase**, con sus propios operadores de concatenación, repetición y transposición, y no una simple secuencia de eventos dentro de una pista.
3. **Modulación.** El cambio de tonalidad preserva la función de cada acorde en lugar de desplazar semitonos a ciegas: modular [I, V, VI, IV] de Do mayor a Sol mayor reconstruye la progresión sobre la nueva tónica.
4. **Conducción de voces con validación de contrapunto.** El compilador distribuye cada acorde entre cuatro voces (SATB) eligiendo las inversiones que minimizan el movimiento melódico, y rechaza los resultados que violan las prohibiciones clásicas de la armonía tradicional: quintas paralelas, octavas paralelas y cruce de voces.

Esta última capacidad es la que define el carácter del proyecto. No se trata de una construcción sintáctica adicional, sino de un algoritmo de optimización y un conjunto de validaciones semánticas propias del dominio, que exigen que el compilador comprenda la estructura vertical de la música y no solo su secuencia horizontal.

4. Construcciones

El lenguaje desarrollado ofrecerá las siguientes construcciones, prestaciones y funcionalidades:

Tipos y constantes

5. Las variables podrán ser de tipo *note*, *interval*, *key*, *scale*, *chord*, *progression*, *voicing*, *integer*, *boolean* o *string*.
6. Las variables podrán ser vectores de alguno de los tipos anteriores, con acceso por índice.
7. Se proveerán constantes propias del dominio: notas (C0 a B8, con alteraciones # y b), grados (I a VII, en mayúscula o minúscula según su calidad, con sufijo ° para los disminuidos), intervalos (P1, m2, M2, m3, M3, P4, A4, d5, P5, m6, M6, m7, M7, P8) y duraciones (*whole*, *half*, *quarter*, *eighth*, *sixteenth*).

Tonalidad y escala

8. Se podrán declarar una o varias tonalidades, cada una con una nota tónica y un modo (*major*, *minor*, *dorian*, *phrygian*, *lydian*, *mixolydian*, *locrian*).
9. Se podrá obtener la escala asociada a una tonalidad y acceder a cualquiera de sus grados.
10. Se podrá declarar una tonalidad en modo estricto (*strict*), en cuyo caso el compilador rechazará todo acorde que contenga notas ajenas a ella.

Acordes y progresiones

11. Se podrán construir acordes por grado sobre una tonalidad (*triad on V of K*, *seventh on ii of K*, *ninth on I of K*), deduciendo el compilador tanto las notas como la calidad del acorde.
12. Se podrán construir acordes enumerando sus notas explícitamente ({C4, E4, G4}).
13. Se podrán construir progresiones como secuencias de grados sobre una tonalidad ([I, V, VI, IV] in K) o como secuencias de acordes concretos.
14. Se podrá invertir un acorde (*invert c by 1*) y arpeggiarlo (*arpeggiate c as eighth*), transformándolo en una secuencia melódica.
15. Se podrá modular una progresión hacia otra tonalidad (*modulate p to G major*), preservando la función armónica de cada acorde, así como hacia relaciones tonales nombradas (*relative minor*, *dominant*, *subdominant*).

Conducción de voces

16. Se podrá distribuir una progresión entre voces (*voice p as SATB*), produciendo un valor de tipo *voicing*. El compilador seleccionará las inversiones que minimicen el movimiento melódico entre acordes consecutivos.
17. Se podrá verificar un *voicing* contra las reglas del contrapunto tradicional (*check v for parallel_fifths*, *parallel_octaves*, *voice_crossing*), reportándose las violaciones como errores de compilación.
18. Se verificará que cada voz permanezca dentro de su tesitura (rango vocal) declarada.

Operadores

19. Se proveerán operaciones aritméticas básicas +, -, * y / sobre enteros.
20. Se proveerán operadores relacionales <, >, =, !=, <= y >=, aplicables entre notas (comparando su altura) y entre intervalos (comparando su amplitud).
21. Se proveerán operaciones lógicas básicas *and*, *or* y *not*.

22. Los operadores + y - estarán sobrecargados sobre el dominio: sumar un intervalo a una nota, a un acorde o a una progresión produce su transposición; sumar dos progresiones las concatena.
23. El operador * entre una progresión y un entero repetirá la progresión esa cantidad de veces.
24. Se proveerá el operador de pertenencia `in`, que determina si una nota pertenece a una escala o a un acorde, produciendo una expresión booleana.
25. Se podrá acceder a las propiedades de los objetos del dominio (`chord.root`, `chord.quality`, `note.octave`, `key.tonic`, `key.mode`, `progression.length`).

Estructura del programa

26. Se proveerán estructuras de control básicas de tipo IF-THEN-ELSE, FOR y WHILE.
27. Se podrán definir procedimientos reutilizables, con parámetros y tipo de retorno, que operen sobre los tipos del dominio.
28. Se podrá emitir el resultado como archivo MIDI o como partitura, especificando el tempo en BPM.
29. Se admitirán comentarios de una línea (`//`) y de bloque (`/* */`).

5. Casos de prueba

5.1. Casos de aceptación

Se proponen los siguientes casos iniciales de prueba de aceptación:

30. Un programa que declare una tonalidad y exporte su escala.
31. Un programa que construya una tríada sobre un grado de una tonalidad.
32. Un programa que construya un acorde de séptima sobre el quinto grado.
33. Un programa que declare un acorde enumerando sus notas explícitamente.
34. Un programa que transporte una nota por un intervalo.
35. Un programa que concatene dos progresiones mediante el operador +.
36. Un programa que repita una progresión mediante el operador *.
37. Un programa que module una progresión a la tonalidad de la dominante.
38. Un programa que module una progresión a su relativo menor.
39. Un programa que recorra los siete grados de una tonalidad con un `for` y construya el acorde correspondiente a cada uno.
40. Un programa que utilice `if-then-else` sobre una condición de pertenencia (`if (E4 in scale of K)`).
41. Un programa que declare un vector de acordes y acceda a sus elementos por índice.
42. Un programa que invierta un acorde.
43. Un programa que arpegie un acorde en corcheas.
44. Un programa que distribuya una progresión a cuatro voces con `voice p as SATB`.
45. Un programa que verifique explícitamente un *voicing* válido contra las reglas de contrapunto.
46. Un programa que utilice un bucle `while` con una condición de comparación entre notas.
47. Un programa que defina un procedimiento y lo invoque.
48. Un programa que exporte una progresión a un archivo MIDI con un tempo dado.

5.2. Casos de rechazo

Además, los siguientes casos de prueba de rechazo:

49. Un programa malformado (falta de punto y coma, o llave sin cerrar).
50. Un programa que utilice una nota inexistente (H4).
51. Un programa que utilice una nota fuera del rango representable (C#9, siendo el rango de octavas 0 a 8).
52. Un programa que referencie un grado inexistente (VIII, dado que una escala diatónica posee siete grados).
53. Un programa que declare un intervalo imposible (P3: las terceras solo pueden ser mayores o menores, nunca justas).
54. Un programa que opere entre tipos de datos incompatibles (`chord + integer`).
55. Un programa que, sobre una tonalidad declarada `strict`, construya un acorde con una nota ajena a ella.
56. Un programa cuyo *voicing* contenga quintas paralelas entre dos voces.
57. Un programa cuyo *voicing* contenga octavas paralelas entre dos voces.
58. Un programa cuyo *voicing* presente cruce de voces (la soprano por debajo de la contralto).
59. Un programa que asigne a una voz una nota fuera de su tesitura.
60. Un programa que utilice una variable no declarada.
61. Un programa que redeclare una variable ya existente en el mismo alcance.
62. Un programa que transporte una nota fuera del rango representable.
63. Un programa que invierta un acorde una cantidad de veces mayor a su cantidad de notas.

6. Ejemplos

En el **Cód. (6.1)** se construye la progresión armónica más frecuente de la música popular — I-V-VI-IV — sobre la tonalidad de Do mayor, se la repite dos veces y se la exporta como archivo MIDI. Nótese que en ningún momento se escriben las notas: el compilador las deduce a partir de la tonalidad y de los grados.

```
// Declarar la tonalidad de trabajo:
key DoMayor = C major;

// Construir la progresión por grados; el compilador deduce las
notas:
progression estrofa = [I, V, VI, IV] in DoMayor;    // Do - Sol - Lam
- Fa

// Repetir la progresión dos veces:
progression tema = estrofa * 2;

// Emitir el resultado:
export tema as midi to "tema.mid" at 120 bpm;
```

Código 6.1: Progresión I-V-VI-IV en Do mayor.

En el **Cód. (6.2)** se recorren los siete grados de una tonalidad menor generando el acorde de cada uno, y se descartan los disminuidos. En la escala menor natural el acorde disminuido se construye sobre el segundo grado (ii°), lo cual el compilador determina automáticamente a partir del modo declarado.

```
key Lam = A minor;

progression diatonicos = [];

for grado = 1 to 7 {
    chord actual = triad on grado of Lam;

    // Descartar los acordes disminuidos, inestables como final de
frase:
    if (actual.quality != diminished) {
        diatonicos = diatonicos + actual;
    }
    else {
        log "Se descartó el grado {1} por ser disminuido", grado;
    }
}

export diatonicos as sheet to "diatonicos.ly";
```

Código 6.2: Generación de los acordes diatónicos de La menor, filtrando los disminuidos.

En el **Cód. (6.3)** se define un procedimiento reutilizable que produce una cadencia auténtica (V-I, el final más característico de la música tonal) para cualquier tonalidad, y se lo aplica antes y después de una modulación a la dominante.

```
// Un procedimiento que arma la cadencia V-I de cualquier tonalidad:
define cadencia(key K) -> progression {
    chord dominante = seventh on V of K;
    chord tonica     = triad on I of K;
    return [dominante, tonica];
}

key origen = C major;

progression obra = cadencia(origen);

// Modular a la dominante preserva la función de cada acorde:
progression puente = modulate obra to dominant of origen; // Sol
mayor

obra = obra + puente + cadencia(origen); // salir y regresar

export obra as midi to "modulacion.mid" at 90 bpm;
```

Código 6.3: Procedimiento que genera cadencias auténticas, con modulación a la dominante y retorno a la tonalidad de origen.

En el **Cód. (6.4)** se distribuye una progresión entre cuatro voces y se la somete a las reglas del contrapunto tradicional. El compilador elige las inversiones que minimizan el movimiento melódico entre acordes consecutivos, y rechaza el programa si el resultado viola alguna de las prohibiciones clásicas.

```
key Sol = G major strict;

progression coral = [I, IV, V, I] in Sol;

// Distribuir cada acorde entre soprano, contralto, tenor y bajo.
// El compilador selecciona las inversiones de menor movimiento:
voicing v = voice coral as SATB;

// Validar contra las reglas de la armonía tradicional:
check v for parallel_fifths, parallel_octaves, voice_crossing;

export v as sheet to "coral.ly";
```

Código 6.4: Conducción de voces a cuatro partes con validación de contrapunto.